

ASSIGNMENT 11.1

Data Structures with AI: Implementing Fundamental Structures

Samala Bharath

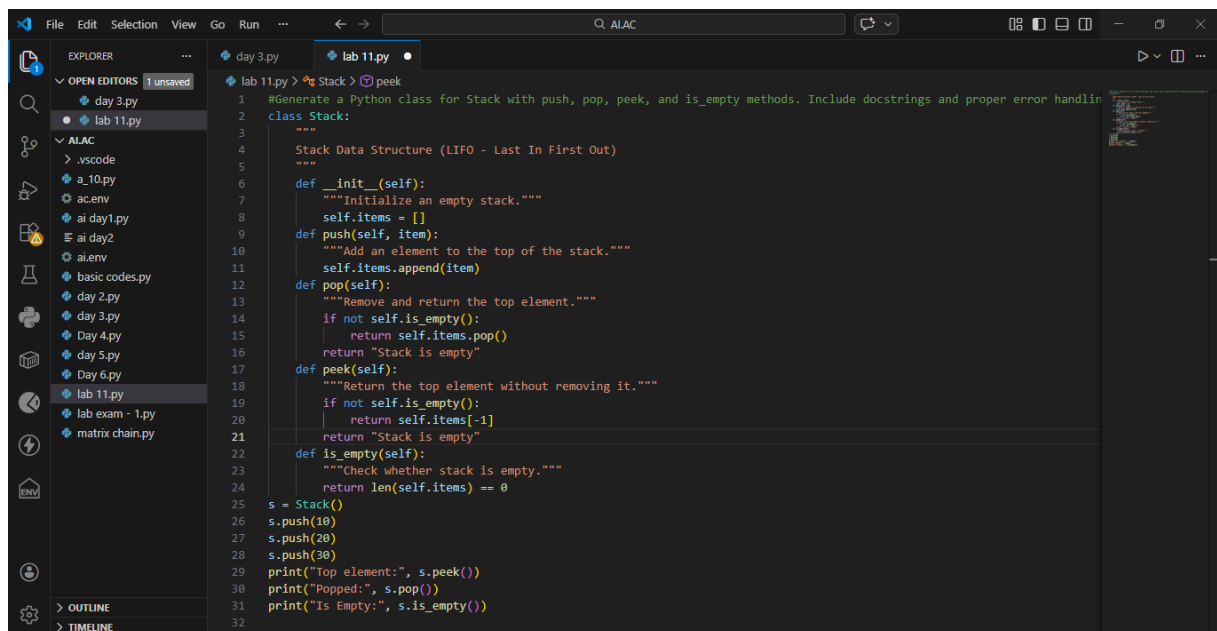
2403A51L17

B-51

Task 1: Stack Implementation

Task: Use AI to generate a Stack class with push, pop, peek, and is_empty methods.

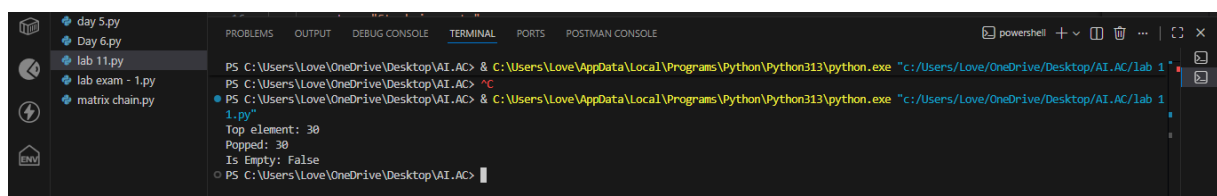
Prompt: Generate a Python class for Stack with push, pop, peek, and is_empty methods. Include docstrings and proper error handling.



The screenshot shows a VS Code editor with a file explorer on the left and a code editor in the center. The file explorer shows a project named 'AIAC' with several files, including 'lab 11.py'. The code editor displays the following Python code for a Stack class:

```
1 #Generate a Python class for Stack with push, pop, peek, and is_empty methods. Include docstrings and proper error handling
2 class Stack:
3     """
4     Stack Data Structure (LIFO - Last In First Out)
5     """
6     def __init__(self):
7         """Initialize an empty stack."""
8         self.items = []
9     def push(self, item):
10        """Add an element to the top of the stack."""
11        self.items.append(item)
12    def pop(self):
13        """Remove and return the top element."""
14        if not self.is_empty():
15            return self.items.pop()
16        return "Stack is empty"
17    def peek(self):
18        """Return the top element without removing it."""
19        if not self.is_empty():
20            return self.items[-1]
21        return "Stack is empty"
22    def is_empty(self):
23        """Check whether stack is empty."""
24        return len(self.items) == 0
25
26 s = Stack()
27 s.push(10)
28 s.push(20)
29 s.push(30)
30 print("Top element:", s.peek())
31 print("Popped:", s.pop())
32 print("Is Empty:", s.is_empty())
```

OUTPUT:



The screenshot shows the VS Code terminal window with the following output:

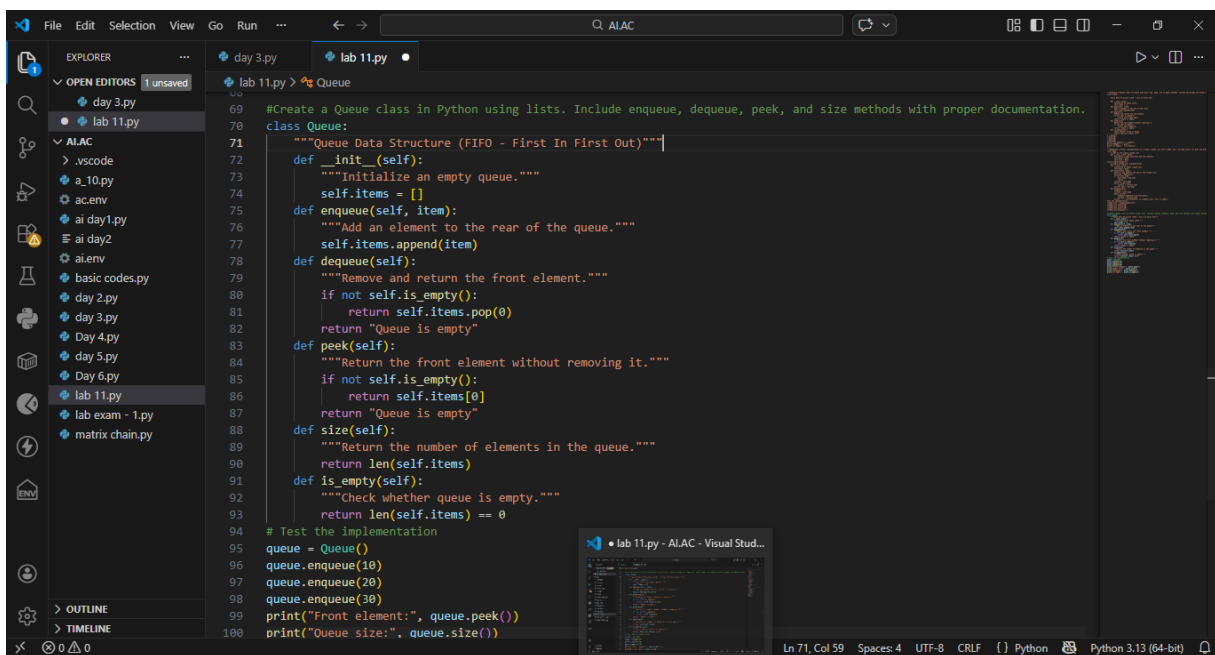
```
PS C:\Users\Love\OneDrive\Desktop\AI.AC> & C:\Users\Love\AppData\Local\Programs\Python\Python313\python.exe "c:/Users/Love/OneDrive/Desktop/AI.AC/lab 11.py"
Top element: 30
Popped: 30
Is Empty: False
PS C:\Users\Love\OneDrive\Desktop\AI.AC>
```

Explanation: A Stack is a linear data structure that follows the LIFO (Last In First Out) principle, where the last element inserted is the first one removed. Operations such as push, pop, and peek are performed at one end called the top. It is commonly used in function calls, undo operations, and expression evaluation.

Task Description #2: Queue Implementation

Task: Use AI to implement a Queue using Python lists.

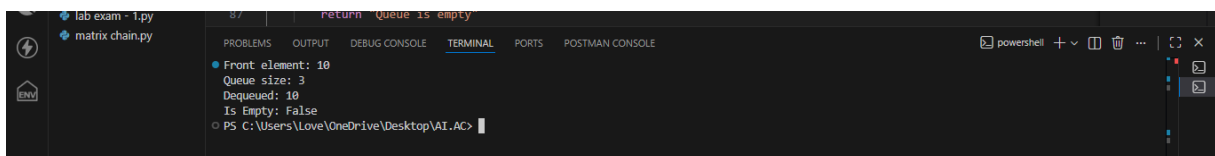
Prompt: Create a Queue class in Python using lists. Include enqueue, dequeue, peek, and size methods with proper documentation.



The screenshot shows a VS Code editor with a file explorer on the left and a code editor in the center. The file explorer shows a project named 'ALAC' with several files, including 'lab 11.py'. The code editor displays the implementation of a Queue class in Python. The class is named 'Queue' and has methods for enqueue, dequeue, peek, and size. The code is as follows:

```
69 #Create a Queue class in Python using lists. Include enqueue, dequeue, peek, and size methods with proper documentation.
70 class Queue:
71     """Queue Data Structure (FIFO - First In First Out)"""
72     def __init__(self):
73         """Initialize an empty queue."""
74         self.items = []
75     def enqueue(self, item):
76         """Add an element to the rear of the queue."""
77         self.items.append(item)
78     def dequeue(self):
79         """Remove and return the front element."""
80         if not self.is_empty():
81             return self.items.pop(0)
82         return "Queue is empty"
83     def peek(self):
84         """Return the front element without removing it."""
85         if not self.is_empty():
86             return self.items[0]
87         return "Queue is empty"
88     def size(self):
89         """Return the number of elements in the queue."""
90         return len(self.items)
91     def is_empty(self):
92         """Check whether queue is empty."""
93         return len(self.items) == 0
94 # Test the implementation
95 queue = Queue()
96 queue.enqueue(10)
97 queue.enqueue(20)
98 queue.enqueue(30)
99 print("Front element:", queue.peek())
100 print("Queue size:", queue.size())
```

OUTPUT:



The screenshot shows the terminal output of the Python code. The output is as follows:

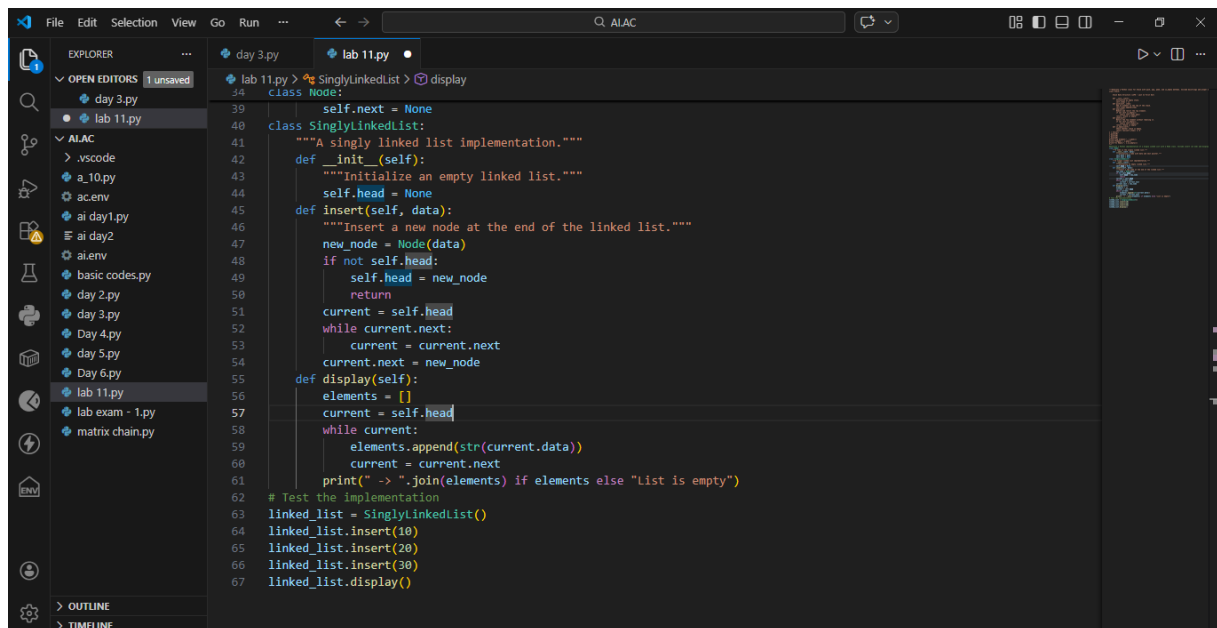
```
Front element: 10
Queue size: 3
Dequeued: 10
Is Empty: False
PS C:\Users\Love\OneDrive\Desktop\AI.AC>
```

Explanation: A Queue is a linear data structure that follows the FIFO (First In First Out) principle. This means the first element inserted is the first one removed.

Task Description #3: Linked List

Task: Use AI to generate a Singly Linked List with insert and display methods.

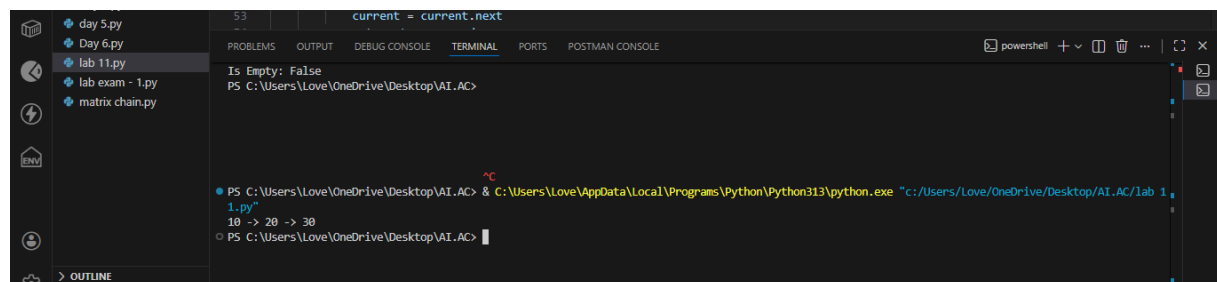
Prompt : Generate a Python implementation of a Singly Linked List with a Node class. Include insert (at end) and display methods with docstrings.



```
File Edit Selection View Go Run ... AIAC
EXPLORER
  OPEN EDITORS 1 unsaved
    day 3.py
    lab 11.py
  AIAC
    .vscode
    a_10.py
    ac.env
    ai day1.py
    ai day2
    ai.env
    basic codes.py
    day 2.py
    day 3.py
    Day 4.py
    day 5.py
    Day 6.py
    lab 11.py
    lab exam - 1.py
    matrix chain.py
  OUTLINE
  TIMELINE

lab 11.py > SinglyLinkedList > display
class Node:
39     self.next = None
40
41 class SinglyLinkedList:
42     """A singly linked list implementation."""
43     def __init__(self):
44         """Initialize an empty linked list."""
45         self.head = None
46     def insert(self, data):
47         """Insert a new node at the end of the linked list."""
48         new_node = Node(data)
49         if not self.head:
50             self.head = new_node
51             return
52         current = self.head
53         while current.next:
54             current = current.next
55         current.next = new_node
56     def display(self):
57         elements = []
58         current = self.head
59         while current:
60             elements.append(str(current.data))
61             current = current.next
62         print("-> ".join(elements) if elements else "List is empty")
63
64 # Test the implementation
65 linked_list = SinglyLinkedList()
66 linked_list.insert(10)
67 linked_list.insert(20)
68 linked_list.insert(30)
69 linked_list.display()
```

OUTPUT:



```
day 5.py
Day 6.py
lab 11.py
lab exam - 1.py
matrix chain.py
OUTLINE

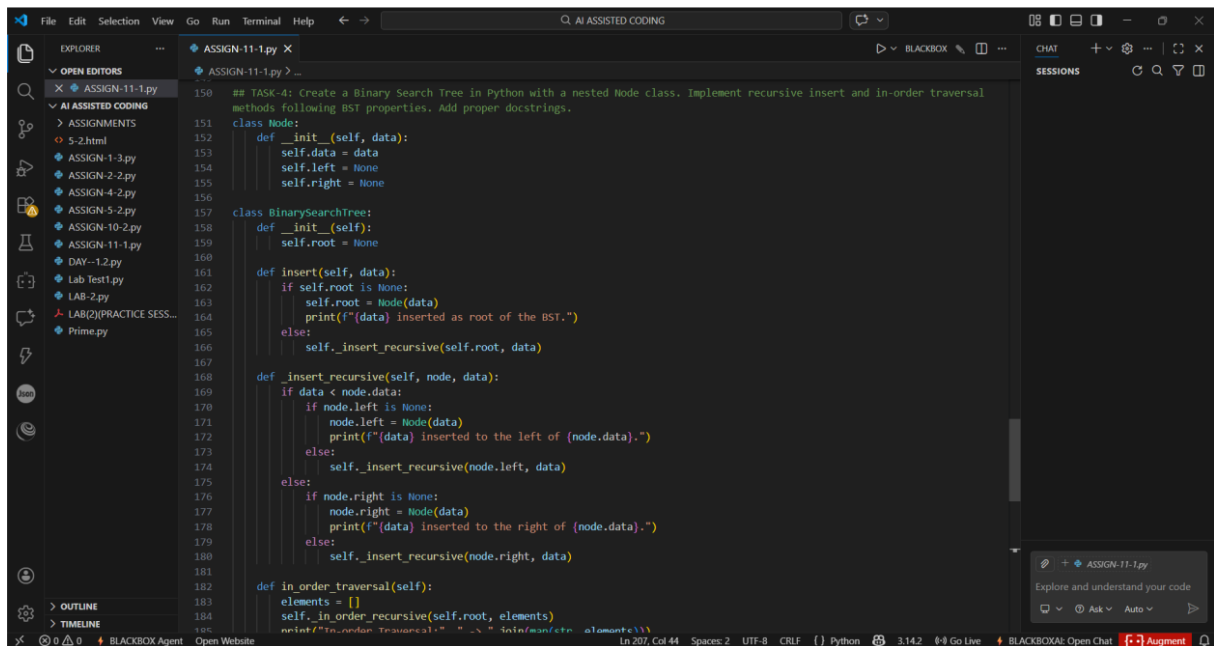
53     current = current.next
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS POSTMAN CONSOLE
Is Empty: False
PS C:\Users\Love\OneDrive\Desktop\AI.AC>
PS C:\Users\Love\OneDrive\Desktop\AI.AC> ^C
PS C:\Users\Love\OneDrive\Desktop\AI.AC> C:\Users\Love\AppData\Local\Programs\Python\Python313\python.exe "c:/Users/Love/OneDrive/Desktop/AI.AC/lab 11.py"
10 -> 20 -> 30
PS C:\Users\Love\OneDrive\Desktop\AI.AC>
```

Explanation: A Singly Linked List is a dynamic data structure where elements (nodes) are connected using pointers. Linked Lists are useful when frequent insertions and deletions are required, as they do not require shifting elements like arrays.

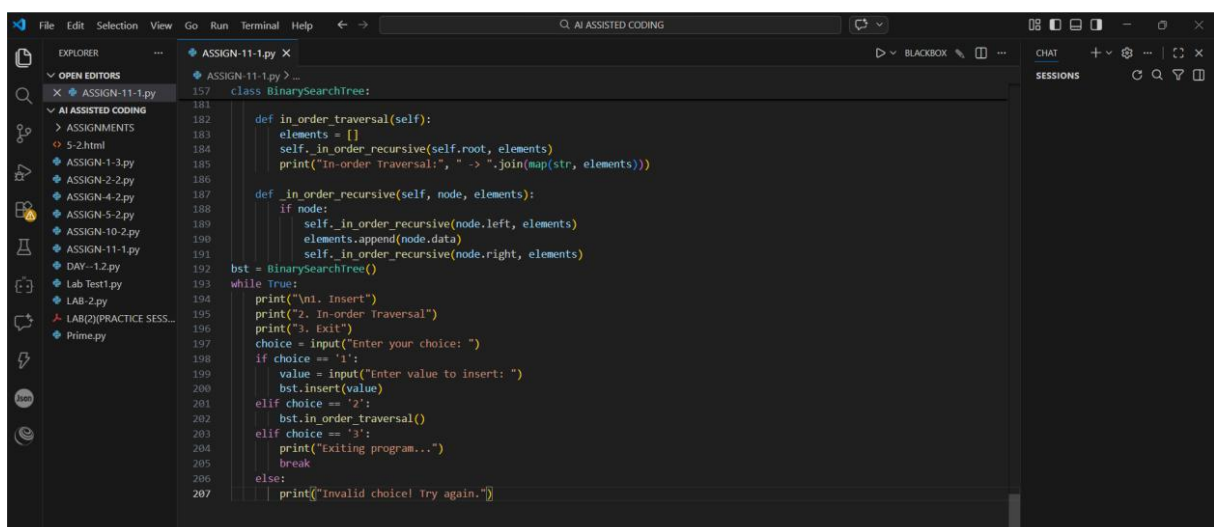
Task Description #4: Binary Search Tree (BST)

Task: Use AI to create a BST with insert and in-order traversal methods.

Prompt: Create a Binary Search Tree in Python with recursive insert and inorder traversal methods. Include proper class structure and documentation.

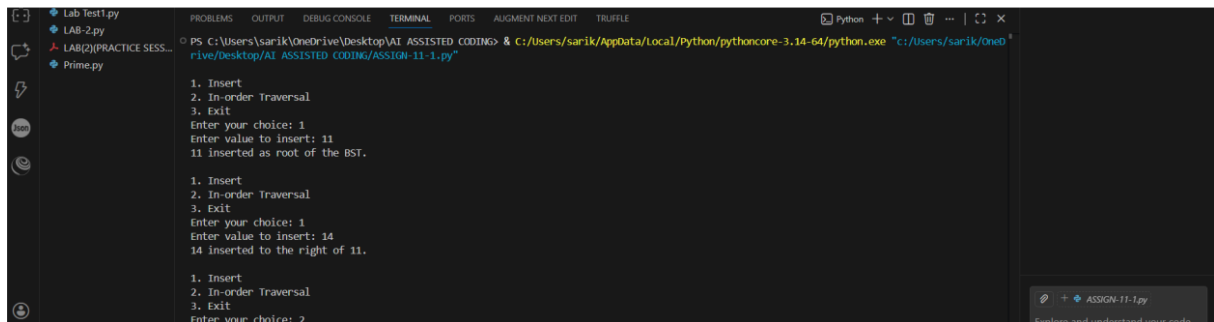


```
150 ## TASK-4: Create a Binary Search Tree in Python with a nested Node class. Implement recursive insert and in-order traversal
151 methods following BST properties. Add proper docstrings.
152
153 class Node:
154     def __init__(self, data):
155         self.data = data
156         self.left = None
157         self.right = None
158
159 class BinarySearchTree:
160     def __init__(self):
161         self.root = None
162
163     def insert(self, data):
164         if self.root is None:
165             self.root = Node(data)
166             print(f"{data} inserted as root of the BST.")
167         else:
168             self._insert_recursive(self.root, data)
169
170     def _insert_recursive(self, node, data):
171         if data < node.data:
172             if node.left is None:
173                 node.left = Node(data)
174                 print(f"{data} inserted to the left of {node.data}.")
175             else:
176                 self._insert_recursive(node.left, data)
177         else:
178             if node.right is None:
179                 node.right = Node(data)
180                 print(f"{data} inserted to the right of {node.data}.")
181             else:
182                 self._insert_recursive(node.right, data)
183
184     def in_order_traversal(self):
185         elements = []
186         self._in_order_recursive(self.root, elements)
187         print("In-order Traversal: ", " ".join(map(str, elements)))
```



```
181
182 class BinarySearchTree:
183
184     def in_order_traversal(self):
185         elements = []
186         self._in_order_recursive(self.root, elements)
187         print("In-order Traversal:", " ".join(map(str, elements)))
188
189     def _in_order_recursive(self, node, elements):
190         if node:
191             self._in_order_recursive(node.left, elements)
192             elements.append(node.data)
193             self._in_order_recursive(node.right, elements)
194
195 bst = BinarySearchTree()
196 while True:
197     print("\n1. Insert")
198     print("2. In-order Traversal")
199     print("3. Exit")
200     choice = input("Enter your choice: ")
201     if choice == '1':
202         value = input("Enter value to insert: ")
203         bst.insert(value)
204     elif choice == '2':
205         bst.in_order_traversal()
206     elif choice == '3':
207         print("Exiting program...")
208         break
209     else:
210         print("Invalid choice! Try again.")
```

OUTPUT:



```
PS C:\Users\sarik\OneDrive\Desktop\AI ASSISTED CODING> & C:\Users\sarik\AppData\Local\Python\pythoncore-3.14-64\python.exe "c:/Users/sarik/OneDrive/Desktop/AI ASSISTED CODING/ASSIGN-11-1.py"

1. Insert
2. In-order Traversal
3. Exit
Enter your choice: 1
Enter value to insert: 11
11 inserted as root of the BST.

1. Insert
2. In-order Traversal
3. Exit
Enter your choice: 1
Enter value to insert: 14
14 inserted to the right of 11.

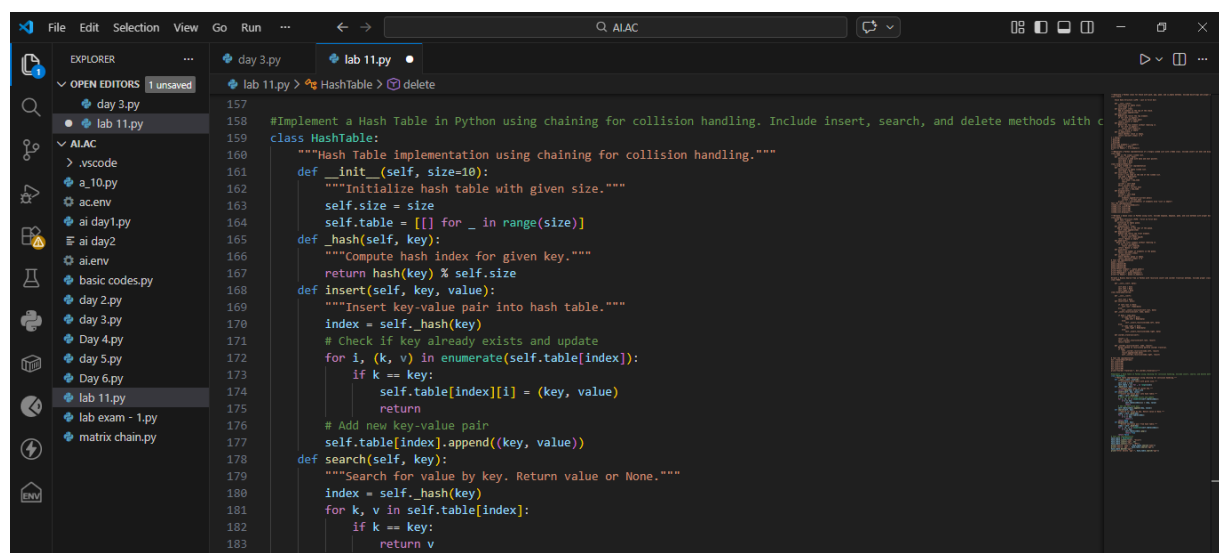
1. Insert
2. In-order Traversal
3. Exit
Enter your choice: 2
```

Explanation: A Binary Search Tree is a hierarchical data structure where the left child contains smaller values and the right child contains larger values than the root. This property makes searching, insertion, and deletion efficient.

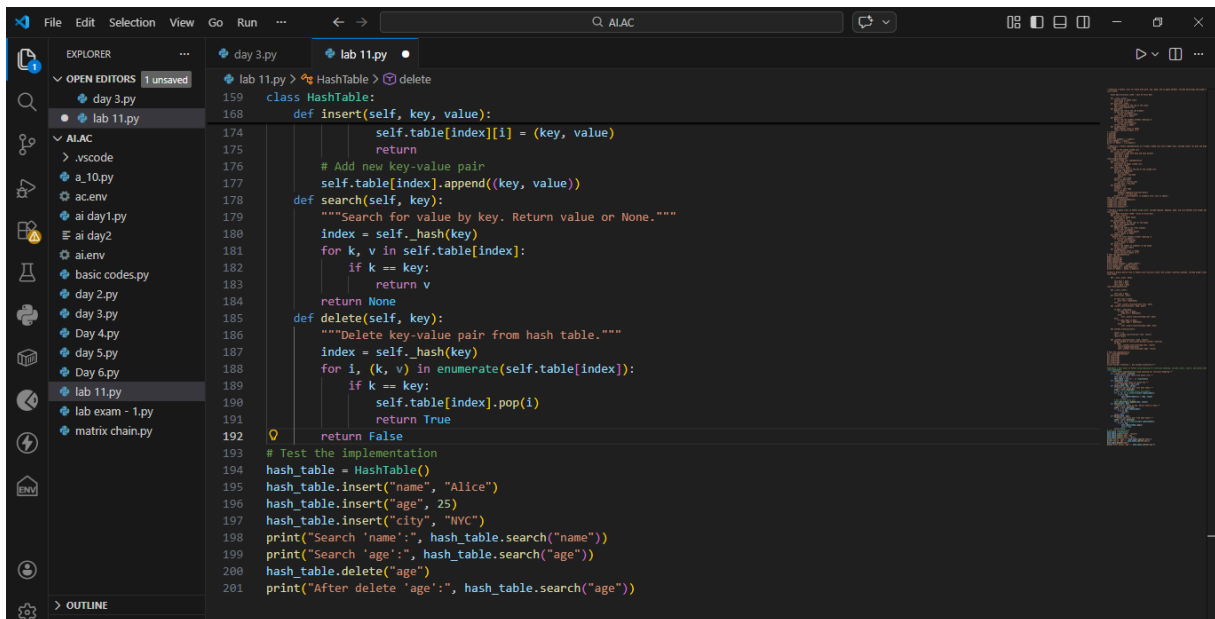
Task Description #5: Hash Table

Task: Use AI to implement a hash table with basic insert, search, and delete methods.

Prompt: Implement a Hash Table in Python using chaining for collision handling. Include insert, search, and delete methods with comments.

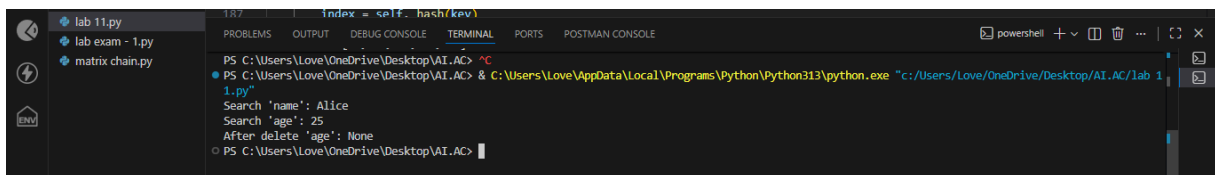


```
File Edit Selection View Go Run ... ALAC
EXPLORER
  OPEN EDITORS 1 unsaved
    day 3.py
    lab 11.py
  ALAC
    .vscode
    a_10.py
    ac.env
    ai day1.py
    ai day2
    ai.env
    basic codes.py
    day 2.py
    day 3.py
    Day 4.py
    day 5.py
    Day 6.py
    lab 11.py
    lab exam - 1.py
    matrix chain.py
  lab 11.py
    HashTable > delete
157
158
159 #Implement a Hash Table in Python using chaining for collision handling. Include insert, search, and delete methods with c
160 class HashTable:
161     """Hash Table implementation using chaining for collision handling."""
162     def __init__(self, size=10):
163         """Initialize hash table with given size."""
164         self.size = size
165         self.table = [[] for _ in range(size)]
166     def _hash(self, key):
167         """Compute hash index for given key."""
168         return hash(key) % self.size
169     def insert(self, key, value):
170         """Insert key-value pair into hash table."""
171         index = self._hash(key)
172         # Check if key already exists and update
173         for i, (k, v) in enumerate(self.table[index]):
174             if k == key:
175                 self.table[index][i] = (key, value)
176                 return
177         # Add new key-value pair
178         self.table[index].append((key, value))
179     def search(self, key):
180         """Search for value by key. Return value or None."""
181         index = self._hash(key)
182         for k, v in self.table[index]:
183             if k == key:
184                 return v
```



```
159 class HashTable:
160     def insert(self, key, value):
174         self.table[index][i] = (key, value)
175         return
176         # Add new key-value pair
177         self.table[index].append((key, value))
178     def search(self, key):
179         """Search for value by key. Return value or None."""
180         index = self._hash(key)
181         for k, v in self.table[index]:
182             if k == key:
183                 return v
184             return None
185     def delete(self, key):
186         """Delete key-value pair from hash table."""
187         index = self._hash(key)
188         for i, (k, v) in enumerate(self.table[index]):
189             if k == key:
190                 self.table[index].pop(i)
191                 return True
192         return False
193 # Test the implementation
194 hash_table = HashTable()
195 hash_table.insert("name", "Alice")
196 hash_table.insert("age", 25)
197 hash_table.insert("city", "NYC")
198 print("Search 'name':", hash_table.search("name"))
199 print("Search 'age':", hash_table.search("age"))
200 hash_table.delete("age")
201 print("After delete 'age':", hash_table.search("age"))
```

OUTPUT:



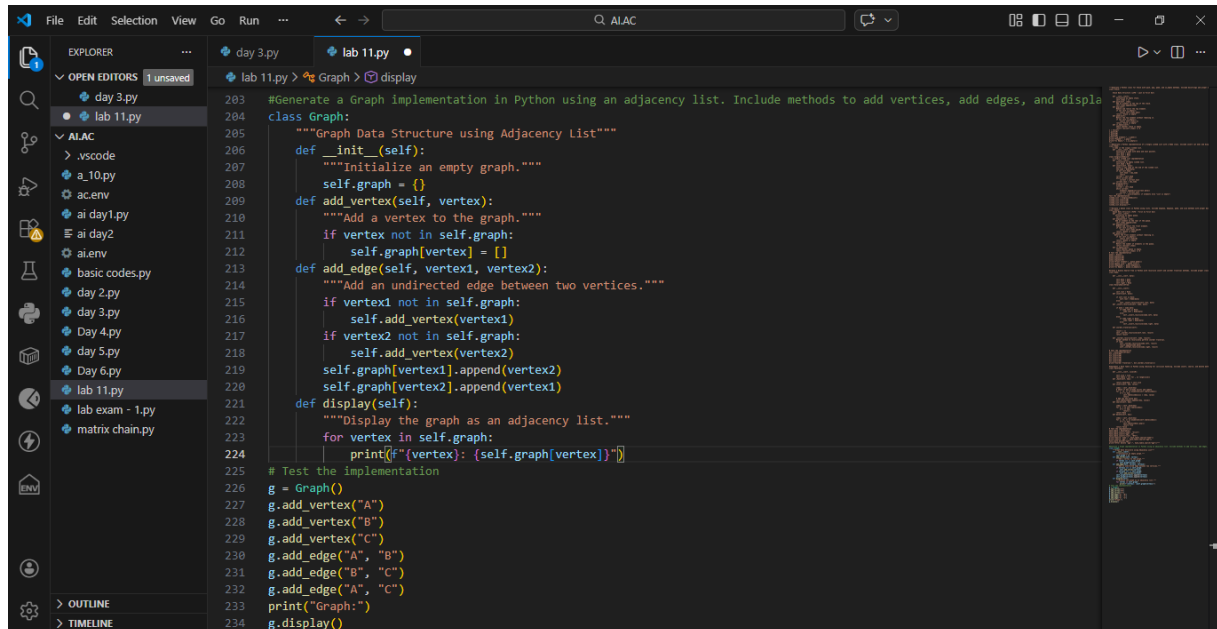
```
PS C:\Users\Love\OneDrive\Desktop\AI.AC> cd C:\Users\Love\OneDrive\Desktop\AI.AC & C:\Users\Love\AppData\Local\Programs\Python\Python313\python.exe "c:/Users/Love/OneDrive/Desktop/AI.AC/lab 11.py"
Search 'name': Alice
Search 'age': 25
After delete 'age': None
PS C:\Users\Love\OneDrive\Desktop\AI.AC>
```

Explanation: A Hash Table stores data in key-value pairs using a hash function to compute an index. It provides fast average-case time complexity for search, insertion, and deletion operations.

Task Description #6: Graph Representation

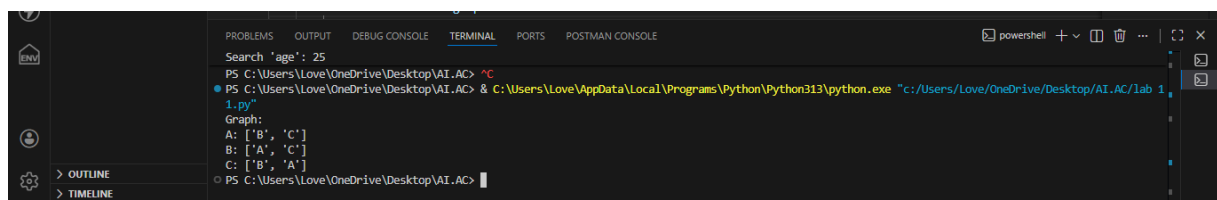
Task: Use AI to implement a graph using an adjacency list.

Prompt: Generate a Graph implementation in Python using an adjacency list. Include methods to add vertices, add edges, and display the graph.



```
203 #Generate a Graph Implementation in Python using an adjacency list. Include methods to add vertices, add edges, and display the graph.
204 class Graph:
205     """Graph Data Structure using Adjacency List"""
206     def __init__(self):
207         """Initialize an empty graph."""
208         self.graph = {}
209     def add_vertex(self, vertex):
210         """Add a vertex to the graph."""
211         if vertex not in self.graph:
212             self.graph[vertex] = []
213     def add_edge(self, vertex1, vertex2):
214         """Add an undirected edge between two vertices."""
215         if vertex1 not in self.graph:
216             self.add_vertex(vertex1)
217         if vertex2 not in self.graph:
218             self.add_vertex(vertex2)
219         self.graph[vertex1].append(vertex2)
220         self.graph[vertex2].append(vertex1)
221     def display(self):
222         """Display the graph as an adjacency list."""
223         for vertex in self.graph:
224             print(f"{vertex}: {self.graph[vertex]}")
225
226 # Test the implementation
227 g = Graph()
228 g.add_vertex("A")
229 g.add_vertex("B")
230 g.add_vertex("C")
231 g.add_edge("A", "B")
232 g.add_edge("B", "C")
233 g.add_edge("A", "C")
234 print("Graph:")
235 g.display()
```

Output:



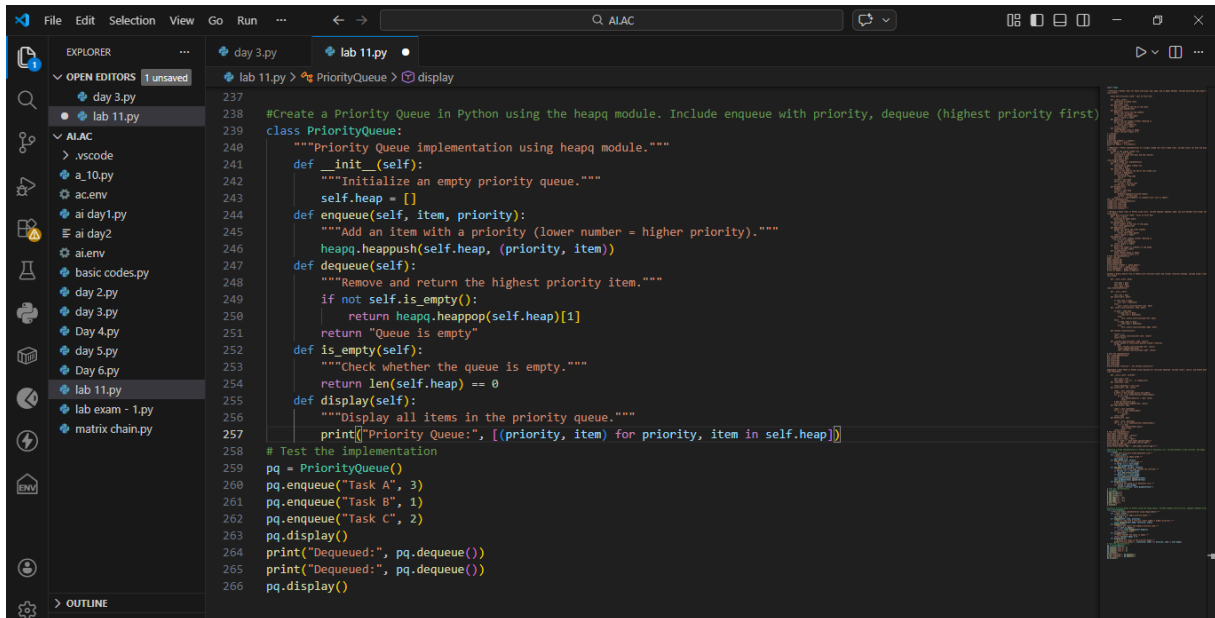
```
Search 'age': 25
PS C:\Users\Love\OneDrive\Desktop\AI.AC> ^C
PS C:\Users\Love\OneDrive\Desktop\AI.AC> & C:\Users\Love\AppData\Local\Programs\Python\Python313\python.exe "c:/Users/Love/OneDrive/Desktop/AI.AC/lab 11.py"
Graph:
A: ['B', 'C']
B: ['A', 'C']
C: ['B', 'A']
PS C:\Users\Love\OneDrive\Desktop\AI.AC>
```

Explanation: A Graph is a non-linear data structure used to represent relationships between entities. It consists of vertices (nodes) and edges (connections).

Task Description #7: Priority Queue

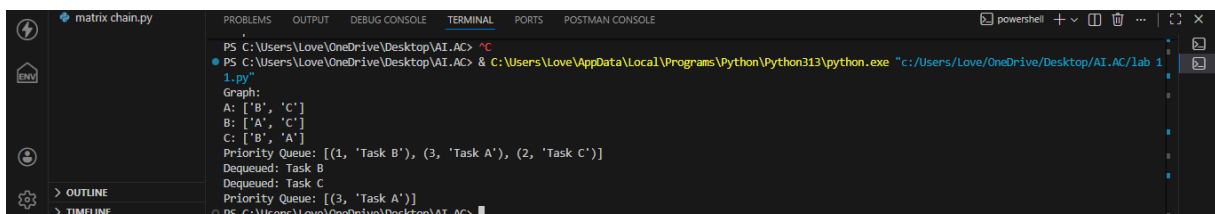
Task: Use AI to implement a priority queue using Python's `heapq` module.

Prompt: Create a Priority Queue in Python using the `heapq` module. Include `enqueue` with priority, `dequeue` (highest priority first), and `display` methods.



```
237
238 #Create a Priority Queue in Python using the heapq module. Include enqueue with priority, dequeue (highest priority first)
239 class PriorityQueue:
240     """Priority Queue implementation using heapq module."""
241     def __init__(self):
242         """Initialize an empty priority queue."""
243         self.heap = []
244     def enqueue(self, item, priority):
245         """Add an item with a priority (lower number = higher priority)."""
246         heapq.heappush(self.heap, (priority, item))
247     def dequeue(self):
248         """Remove and return the highest priority item."""
249         if not self.is_empty():
250             return heapq.heappop(self.heap)[1]
251         return "Queue is empty"
252     def is_empty(self):
253         """Check whether the queue is empty."""
254         return len(self.heap) == 0
255     def display(self):
256         """Display all items in the priority queue."""
257         print("Priority Queue:", [(priority, item) for priority, item in self.heap])
258 # Test the implementation
259 pq = PriorityQueue()
260 pq.enqueue("Task A", 3)
261 pq.enqueue("Task B", 1)
262 pq.enqueue("Task C", 2)
263 pq.display()
264 print("Dequeued:", pq.dequeue())
265 print("Dequeued:", pq.dequeue())
266 pq.display()
```

Output:



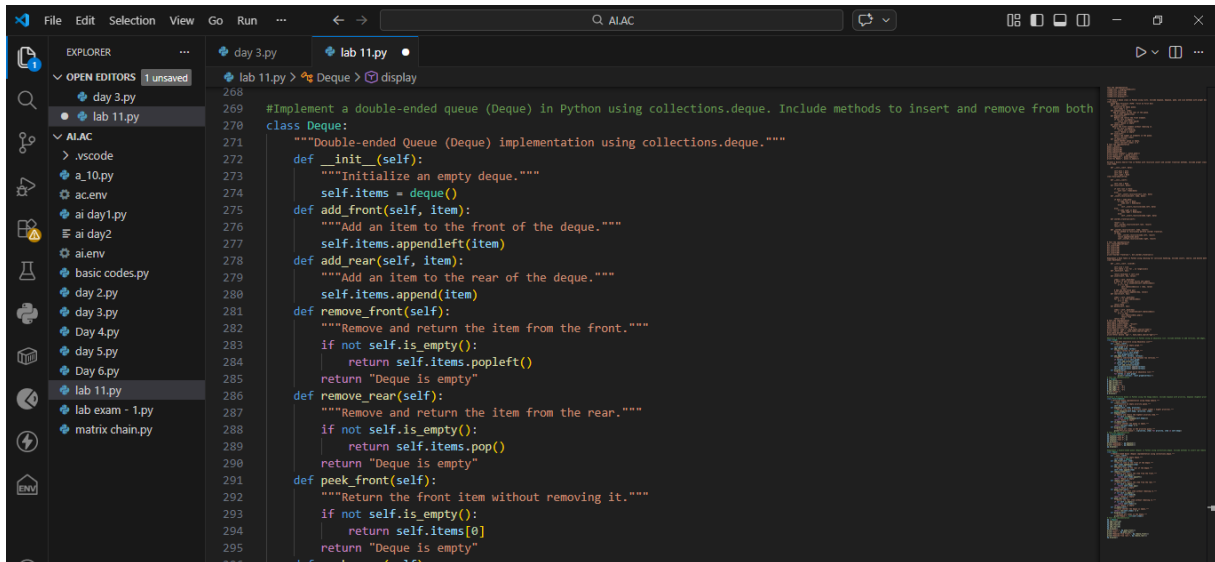
```
PS C:\Users\Love\OneDrive\Desktop\AI.AC> ^C
PS C:\Users\Love\OneDrive\Desktop\AI.AC> & C:\Users\Love\AppData\Local\Programs\Python\Python313\python.exe -c "c:/Users/Love/OneDrive/Desktop/AI.AC/lab 1
1.py"
Graph:
A: ['B', 'C']
B: ['A', 'C']
C: ['B', 'A']
Priority Queue: [(1, 'Task B'), (3, 'Task A'), (2, 'Task C')]
Dequeued: Task B
Dequeued: Task C
Priority Queue: [(3, 'Task A')]
PS C:\Users\Love\OneDrive\Desktop\AI.AC>
```

Explanation: A Priority Queue is a special type of queue where elements are removed based on priority rather than order of insertion. Higher priority elements are processed first. It is typically implemented using a heap for efficiency.

Task Description #8 – Deque

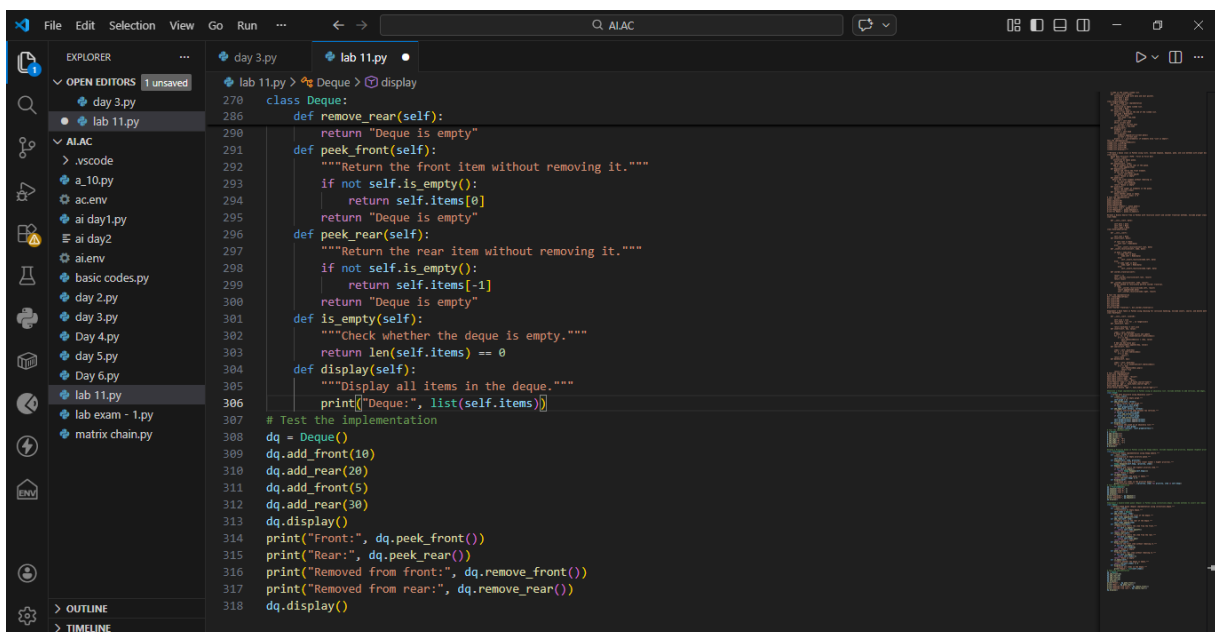
Task: Use AI to implement a double-ended queue using `collections.deque`.

Prompt: Implement a double-ended queue (Deque) in Python using `collections`, `deque`. Include methods to insert and remove from both ends with documentation.



The screenshot shows a VS Code editor window with the Explorer sidebar on the left, the main editor in the center, and the Output sidebar on the right. The Explorer sidebar shows a project structure with files like `day 3.py`, `lab 11.py`, and `lab exam - 1.py`. The main editor displays the implementation of a `Deque` class in `lab 11.py`. The code includes a docstring, an `__init__` method, and methods for adding and removing items from both ends, as well as peeking and displaying the deque.

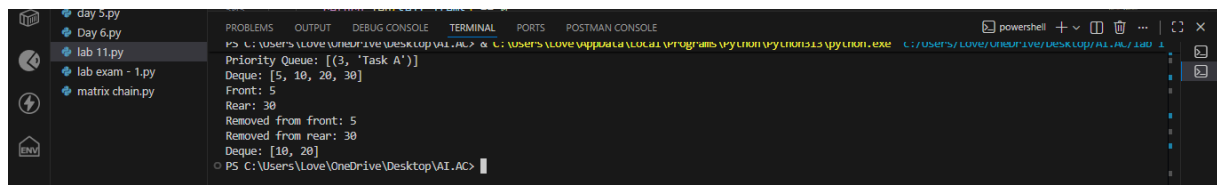
```
268 #Implement a double-ended queue (Deque) in Python using collections.deque. Include methods to insert and remove from both
269
270 class Deque:
271     """Double-ended Queue (Deque) implementation using collections.deque."""
272     def __init__(self):
273         """Initialize an empty deque."""
274         self.items = deque()
275     def add_front(self, item):
276         """Add an item to the front of the deque."""
277         self.items.appendleft(item)
278     def add_rear(self, item):
279         """Add an item to the rear of the deque."""
280         self.items.append(item)
281     def remove_front(self):
282         """Remove and return the item from the front."""
283         if not self.is_empty():
284             return self.items.popleft()
285         return "Deque is empty"
286     def remove_rear(self):
287         """Remove and return the item from the rear."""
288         if not self.is_empty():
289             return self.items.pop()
290         return "Deque is empty"
291     def peek_front(self):
292         """Return the front item without removing it."""
293         if not self.is_empty():
294             return self.items[0]
295         return "Deque is empty"
296     def peek_rear(self):
297         """Return the rear item without removing it."""
298         if not self.is_empty():
299             return self.items[-1]
300         return "Deque is empty"
301     def is_empty(self):
302         """Check whether the deque is empty."""
303         return len(self.items) == 0
304     def display(self):
305         """Display all items in the deque."""
306         print("Deque:", list(self.items))
307
308 # Test the implementation
309 dq = Deque()
310 dq.add_front(10)
311 dq.add_rear(20)
312 dq.add_front(5)
313 dq.add_rear(30)
314 dq.display()
315 print("Front:", dq.peek_front())
316 print("Rear:", dq.peek_rear())
317 print("Removed from front:", dq.remove_front())
318 print("Removed from rear:", dq.remove_rear())
319 dq.display()
```



The screenshot shows the continuation of the `Deque` class implementation in `lab 11.py`. The code includes methods for peeking at the front and rear, checking if the deque is empty, and displaying the deque. It also includes a test section that creates a `Deque` object, adds items to both ends, and performs various operations to verify the implementation.

```
270 class Deque:
271     """Double-ended Queue (Deque) implementation using collections.deque."""
272     def __init__(self):
273         """Initialize an empty deque."""
274         self.items = deque()
275     def add_front(self, item):
276         """Add an item to the front of the deque."""
277         self.items.appendleft(item)
278     def add_rear(self, item):
279         """Add an item to the rear of the deque."""
280         self.items.append(item)
281     def remove_front(self):
282         """Remove and return the item from the front."""
283         if not self.is_empty():
284             return self.items.popleft()
285         return "Deque is empty"
286     def remove_rear(self):
287         """Remove and return the item from the rear."""
288         if not self.is_empty():
289             return self.items.pop()
290         return "Deque is empty"
291     def peek_front(self):
292         """Return the front item without removing it."""
293         if not self.is_empty():
294             return self.items[0]
295         return "Deque is empty"
296     def peek_rear(self):
297         """Return the rear item without removing it."""
298         if not self.is_empty():
299             return self.items[-1]
300         return "Deque is empty"
301     def is_empty(self):
302         """Check whether the deque is empty."""
303         return len(self.items) == 0
304     def display(self):
305         """Display all items in the deque."""
306         print("Deque:", list(self.items))
307
308 # Test the implementation
309 dq = Deque()
310 dq.add_front(10)
311 dq.add_rear(20)
312 dq.add_front(5)
313 dq.add_rear(30)
314 dq.display()
315 print("Front:", dq.peek_front())
316 print("Rear:", dq.peek_rear())
317 print("Removed from front:", dq.remove_front())
318 print("Removed from rear:", dq.remove_rear())
319 dq.display()
```

Output:



```
PS C:\Users\Love\OneDrive\Desktop\AT.AC> python C:\Users\Love\AppData\Local\Programs\Python\Python313\python.exe C:/Users/Love/OneDrive/Desktop/AT.AC/lab 1
Priority Queue: [(3, 'Task A')]
Deque: [5, 10, 20, 30]
Front: 5
Rear: 30
Removed from front: 5
Removed from rear: 30
Deque: [10, 20]
PS C:\Users\Love\OneDrive\Desktop\AT.AC>
```

Explanation: A Deque (Double Ended Queue) allows insertion and deletion of elements from both the front and rear ends.